Attention at the Atomic Level

How Q, K, V Build One Prediction, Operation by Operation

> This document walks through every single arithmetic operation that happens when **one attention head** reads from a key/value cache and produces an output vector. Same audience and same style as the *Perceptron — Forward and Backward, Atom by Atom* handout. By the end you should be able to recompute every number in the tables on paper, and recognise exactly which lines of `MultiHeadAttention.forward` they correspond to.

Companion to: the perceptron handout (autograd basics) and the Tiny GPT 16-point walkthrough (high-level architecture). This sits between them — atomic mechanics applied specifically to attention.

---

1. What is attention trying to do?

The model wants to predict the next character. To do that well, it needs to **look back** at characters it has already seen. Attention is the mechanism that lets it look back **selectively**: not all past characters matter equally for the next prediction.

The picture in plain English:

> For each current position, the model produces a **query** vector — a "probe" asking which past positions matter for what comes next. Every past position has already produced a **key** (an advertisement of what it knows) and a **value** (the information itself); the model takes the dot product of the query against every past key to get one similarity score per past position. Softmax turns those scores into weights that sum to 1 — bigger dot products become bigger weights, smoothly, not in an all-or-nothing match/no-match way. The attention output is the weighted blend of past values: positions whose keys align with the query dominate the blend; positions whose keys point elsewhere contribute almost nothing.

Attention is therefore three things glued together:

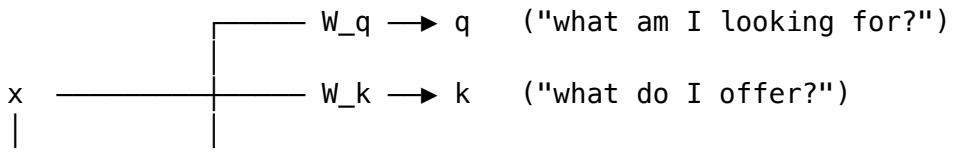
1. Project the current input into three different vectors — **query**, **key**, **value**.
2. Score the current query against every past key (using a dot product and a softmax).
3. Take a weighted sum of past values using those scores as weights.

Today we trace one full attention computation with concrete numbers so all three pieces become arithmetic.

---

2. The Q, K, V trio

Three matrices, all the same shape, all multiplied into the same input `x`:



```
│              └──── W_v ──▶ v   ("what would I contribute?")
│
└─ the current position's hidden vector (one number per dim)
```

The query (q), key (k), and value (v) are all **projections of the same input**. The matrices `W_q`, `W_k`, `W_v` are learned — they start random and training adjusts them so the projections become useful.

Once we have `(q, k, v)`:

- `k` and `v` get **appended to the cache** so future positions can attend to this one. (`k` joins `keys_cache`, `v` joins `values_cache`.)
- `q` is used **right now** to compare against every key in the cache (including the one we just appended).

---

3. The example we'll trace

We pick small concrete numbers so every line is checkable.

Setup

- We're at some position partway through a sequence. **Two past positions** are already in the cache from prior forward calls.
- Embedding width (here equal to `head_dim`): **2**.
- One attention head only. (Multi-head is just "the same thing N times in parallel" — see §7.)

Current input (the hidden vector at this position):

Component	Value
x[0]	1.0
x[1]	2.0

Projection matrices (chosen to keep the numbers clean — in a real trained model these would be learned):

```
W_q = [[1.0,  0.0],        W_k = [[ 0.5,  0.5],        W_v = [[1.0, 0.0],
       [0.0,  1.0]]               [-0.5,  0.5]]               [0.0, 1.0]]
```

`W_q` and `W_v` are identity matrices. That's a deliberate pedagogical choice — it means q and v are literally x, so all the "interesting" projection arithmetic happens through W_k. (In a real model these would all be random/learned.)

Cache contents (2 past timesteps already accumulated):

timestep	`keys_cache[t]`	`values_cache[t]`
0	[1.0, 1.0]	[1.0, 0.0]
1	[2.0, 0.0]	[0.0, 1.0]

By the end of the forward pass below, the cache will have grown to three entries.

---

4. Forward, atom by atom

The computation breaks into six stages. Each stage adds Value nodes to the computation graph; each row in the tables below is one new node.

4.1 Project x into q, k, v

The projection `y = W · x` for a 2-output Linear layer is:

```
y[0] = W[0][0]·x[0] + W[0][1]·x[1]      ← 2 multiplies + 1 add
y[1] = W[1][0]·x[0] + W[1][1]·x[1]      ← 2 multiplies + 1 add
```

That's six Value operations per matrix, eighteen for q, k, v together.

#	Code	Calculation	New value
1	`q[0] = W_q[0][0]*x[0] + W_q[0][1]*x[1]`	1.0·1.0 + 0.0·2.0	**1.0**
2	`q[1] = W_q[1][0]*x[0] + W_q[1][1]*x[1]`	0.0·1.0 + 1.0·2.0	**2.0**
3	`k[0] = W_k[0][0]*x[0] + W_k[0][1]*x[1]`	0.5·1.0 + 0.5·2.0	**1.5**
4	`k[1] = W_k[1][0]*x[0] + W_k[1][1]*x[1]`	–0.5·1.0 + 0.5·2.0	**0.5**
5	`v[0] = W_v[0][0]*x[0] + W_v[0][1]*x[1]`	1.0·1.0 + 0.0·2.0	**1.0**
6	`v[1] = W_v[1][0]*x[0] + W_v[1][1]*x[1]`	0.0·1.0 + 1.0·2.0	**2.0**

(Each row is two multiply Values plus one add Value internally — six new nodes per row, eighteen across this table.)

Result so far:

```
q = [1.0, 2.0]      k = [1.5, 0.5]      v = [1.0, 2.0]
```

4.2 Append k, v to the cache

This is bookkeeping — no new Value nodes are created, the lists just grow by one entry each.

```
keys_cache   = [[1.0, 1.0],  [2.0, 0.0],  [1.5, 0.5]]
                 timestep 0   timestep 1   timestep 2 (just appended)

values_cache = [[1.0, 0.0],  [0.0, 1.0],  [1.0, 2.0]]
                 timestep 0   timestep 1   timestep 2 (just appended)
```

There are now **three** keys to score against.

4.3 Dot-product scores

For each past timestep `t`, compute `q · keys_cache[t]`:

#	Code	Calculation	New value
1	`dot_0 = q[0]·k_cache[0][0] + q[1]·k_cache[0][1]`	1.0·1.0 + 2.0·1.0	**3.0**
2	`dot_1 = q[0]·k_cache[1][0] + q[1]·k_cache[1][1]`	1.0·2.0 + 2.0·0.0	**2.0**
3	`dot_2 = q[0]·k_cache[2][0] + q[1]·k_cache[2][1]`	1.0·1.5 + 2.0·0.5	**2.5**

(Each row is 2 multiply Values + 1 add Value — three new nodes per row, nine across this table.)

A larger dot product means the query and that key point in similar directions, i.e. the model considers this past token more relevant. Right now the largest is `dot_0 = 3.0`, so past timestep 0 is the front-runner.

4.4 Scale by √head_dim

Why divide? Without this scaling, the dot products grow proportional to head_dim, the softmax saturates, and gradients vanish for most positions. Dividing by √`head_dim` keeps the score variance ~1.

With `head_dim = 2`, the divisor is √2 ≈ `1.4142`.

#	Code	Calculation	New value
1	`score_0 = dot_0 / √2`	3.0 / 1.4142	**2.1213**
2	`score_1 = dot_1 / √2`	2.0 / 1.4142	**1.4142**
3	`score_2 = dot_2 / √2`	2.5 / 1.4142	**1.7678**

(Each division is one Value node — three new nodes total.†)

†Tiny implementation note: in the actual `Value` class `a / b` is defined as `a * (b ** -1)`, so each division technically creates three nodes internally (a wrapped float, the reciprocal, and the product). For clarity we count it as one operation here.

4.5 Softmax

Softmax converts the three raw scores into a probability distribution over the past timesteps. The trick: **subtract the maximum first** so the largest exponent is `exp(0) = 1` instead of something like `exp(800)` which would overflow. This is a numerical-stability technique that does not change the result.

```
max_val = max(scores)            # plain max — no Value created
exps    = [(s - max_val).exp() for s in scores]
total   = exps[0] + exps[1] + exps[2]
probs   = [e / total for e in exps]
```

max_val = 2.1213 (the largest score, from timestep 0).

The Value-creating steps:

#	Code	Calculation	New value
1	`s0 - max_val`	2.1213 – 2.1213	0.0
2	`s1 - max_val`	1.4142 – 2.1213	–0.7071
3	`s2 - max_val`	1.7678 – 2.1213	–0.3536
4	`e0 = exp(s0 - max_val)`	exp(0)	**1.0000**
5	`e1 = exp(s1 - max_val)`	exp(–0.7071)	**0.4931**
6	`e2 = exp(s2 - max_val)`	exp(–0.3536)	**0.7022**
7	`total = e0 + e1`	1.0000 + 0.4931	1.4931
8	`total = total + e2`	1.4931 + 0.7022	2.1953
9	`p0 = e0 / total`	1.0000 / 2.1953	**0.4555**
10	`p1 = e1 / total`	0.4931 / 2.1953	**0.2246**
11	`p2 = e2 / total`	0.7022 / 2.1953	**0.3199**

Sanity check: `p0 + p1 + p2 = 0.4555 + 0.2246 + 0.3199 = 1.0000` (sums to 1.0 as required).

(Eleven new Value nodes for the softmax. Step 7 holds an intermediate sum used only inside the computation; the final attention weights are p0, p1, p2.)

4.6 Weighted sum of past values

The attention output is `Σ_t  p_t · v_cache[t]`. We compute it element-wise across the two output dimensions.

#	Code	Calculation	New value
1	`out[0] = p0·v_cache[0][0]`	0.4555 · 1.0	0.4555
2	`out[0] += p1·v_cache[1][0]`	0.4555 + 0.2246·0.0	0.4555
3	`out[0] += p2·v_cache[2][0]`	0.4555 + 0.3199·1.0	**0.7754**
4	`out[1] = p0·v_cache[0][1]`	0.4555 · 0.0	0.0000
5	`out[1] += p1·v_cache[1][1]`	0.0000 + 0.2246·1.0	0.2246
6	`out[1] += p2·v_cache[2][1]`	0.2246 + 0.3199·2.0	**0.8643**

Final attention output: `out = [0.7754, 0.8643]`.

(For each output dimension: 3 multiplies + 2 adds = 5 Value nodes. Ten new nodes across this table. In the real code each += is `acc = acc + new_term` — a fresh Value, not in-place.)

4.7 Tally

Stage	New Value nodes
4.1 — q, k, v projections (3 × 2 outputs)	18
4.2 — cache append	0
4.3 — three dot products	9
4.4 — three scalings	3
4.5 — softmax	11
4.6 — weighted sum (2 output dims)	10

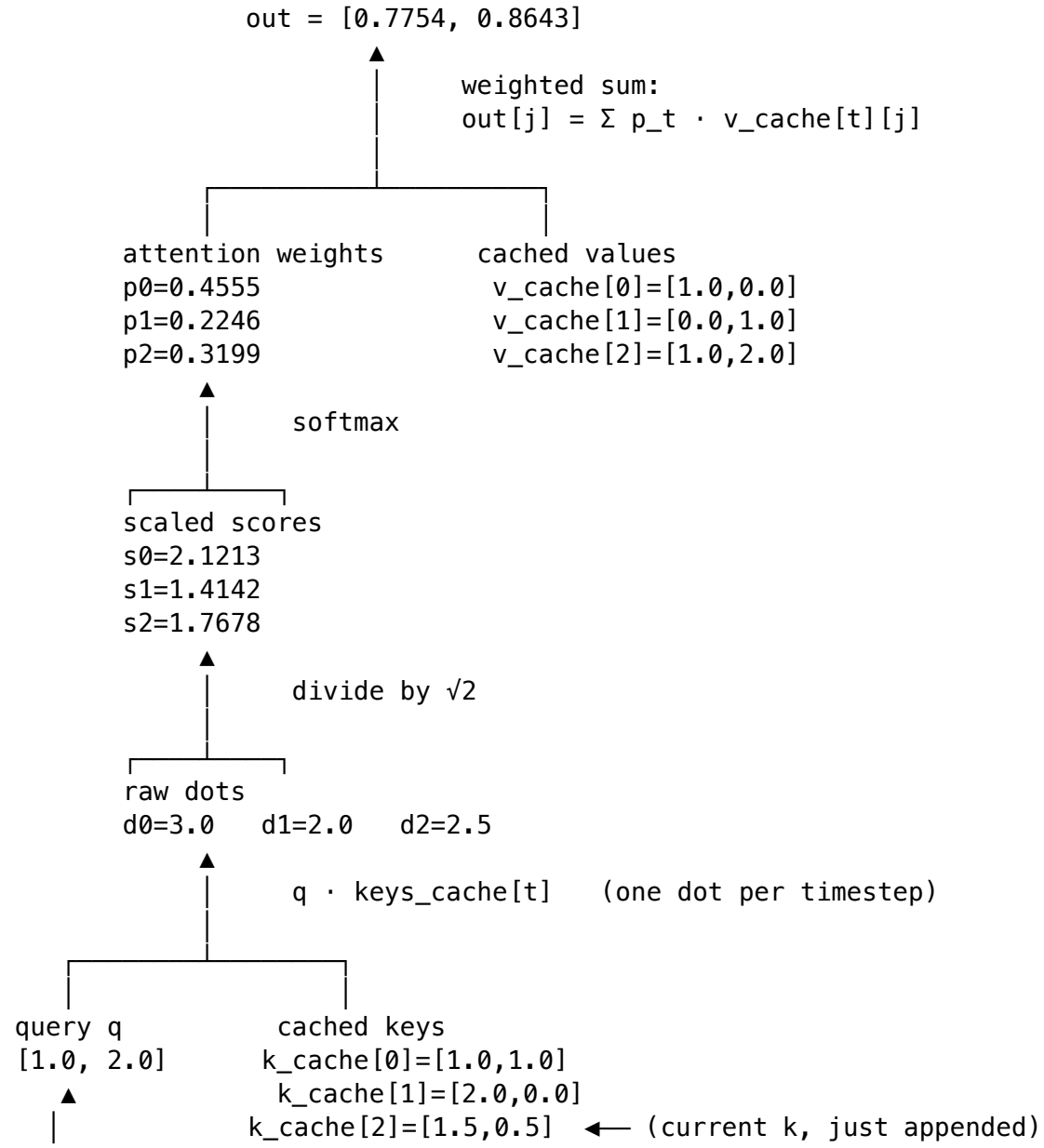
Stage	New Value nodes
Total	**51**

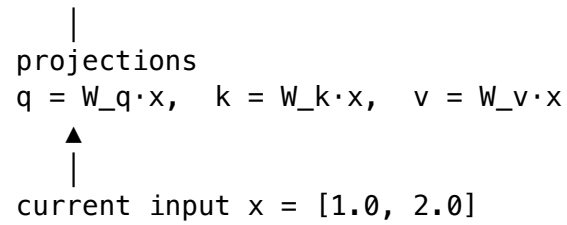
Fifty-one new Value nodes built one prediction's worth of attention for one head. The autograd engine you saw in the perceptron handout will, on the way back, walk every one of them and assign each its gradient.

---

5. The computation graph

A node-by-node tree of all 51 Values would be unreadable. Here is the **flow** instead — the layers of computation, with the key values filled in:





The horizontal arrows in stage **"weighted sum"** mean every output dimension reads from every (attention weight, value) pair — that's the "mixing" attention does.

---

6. What the model "decided"

The final attention weights are:

past timestep	key	attention weight	interpretation
0	[1.0, 1.0]	**0.4555**	this past token gets ~46% of the attention
1	[2.0, 0.0]	0.2246	this past token gets ~22% — actively suppressed
2	[1.5, 0.5]	0.3199	the current position's own k gets ~32%

Why did timestep 0 win? Look at its key `[1.0, 1.0]` versus the query `q = [1.0, 2.0]`. Both have a strong positive second component, so the dot product piles up. Timestep 1's key `[2.0, 0.0]` has a zero in the second component — exactly the dimension where q is largest — so it can't earn weight there.

The output `[0.7754, 0.8643]` is a blend: mostly drawn from `v_cache[0] = [1.0, 0.0]` and `v_cache[2] = [1.0, 2.0]`, with a small contribution from `v_cache[1] = [0.0, 1.0]`. The model has "decided" to mix information from past tokens 0 and 2 to inform whatever comes next.

That is what attention does, every step, in every layer: **read past positions selectively**.

---

7. Multi-head, in one paragraph

A real transformer doesn't run attention once on the whole embedding. It splits the embedding into `n_head` slices of `head_dim` each, runs the entire procedure above **independently per slice**, then concatenates the per-head outputs back into a single vector and projects the result one more time (through `attn_wo`). Different heads learn different attention patterns — one head might learn "look at the very last token", another might learn "look back to the beginning of the name". Nothing in §4 changes per head. It just happens `n_head` times in parallel.

In the tiny GPT used in this course: `n_embd = 16`, `n_head = 4`, so `head_dim = 4`. The §4 trace would run **four times** with different slices and different projection rows; the four outputs (each of length 4) would concatenate back to length 16; one more Linear would mix them.

8. Backward, sketched

The perceptron handout traced every chain-rule step by hand. For attention we **won't** trace all of it — softmax's derivative is intricate (each output `p_i` depends on every input `s_j` via `p_i(δ_{ij} − p_j)`), so a full backward through these 51 Values is roughly 150 chain-rule applications. Beyond what fits on a handout.

But the **mechanism** is identical to the perceptron's. Here is the gradient flow at a block level:

```
                    dL/d(out)          ← given by the next layer up
                       │
                       ▼
                dL/d(attention weights)   ← out = Σ p_t · v_t
                    │           so dL/dp_t = Σ_j (dL/dout[j]) · v_cache[t][j]
                       ▼
                dL/d(scaled scores)       ← softmax backward
                     │            (p_i partial derivative; see formula below)
                       ▼
                dL/d(raw dots)             ← divide by √d_head
                          │              dL/ddot_t = (dL/dscore_t) / √d_head
                          ▼
          dL/d(q)  and  dL/d(k_cache[t])  ← d/dq (q·k_t) = k_t,  d/dk_t (q·k_t) = q
                          │
                          ▼
                dL/d(W_q), dL/d(W_k), dL/d(W_v)  ← Linear backward
                                  (same pattern as the perceptron's W gradient)
```

The softmax derivative, for completeness:

```
∂p_i / ∂s_j  =  p_i · (δ_{ij} − p_j)
                                    where δ_{ij} = 1 if i == j else 0
```

You don't need this formula to *use* the model — `Value.backward()` knows it because softmax is built out of `exp`, `add`, and `div`, each of which has its derivative recorded at construction time. This is exactly the point of autograd: **you don't have to write softmax's backward; the engine derives it from the forward graph.**

If you want to convince yourself, pick one of the `(p_i, s_j)` pairs and trace it through the 11 nodes in §4.5. You'll end up with the formula above.

---

9. Where this lives in the code

This entire walkthrough corresponds to lines inside one function: `MultiHeadAttention.forward` in `microgpt-edu/microgpt/model.py`.

```python
def forward(self, x, keys_cache, values_cache, verbose=False):

    # §4.1 in this handout
    q = self.query.forward(x)
```

```
    k = self.key.forward(x)
    v = self.value.forward(x)

    # §4.2 in this handout
    keys_cache.append(k)
    values_cache.append(v)
    t_plus_1 = len(keys_cache)

    sqrt_head_dim = self.head_dim ** 0.5

    for h in range(self.n_head):                    # §7 — one iteration per head
        head_start = h * self.head_dim
        q_h = q[head_start : head_start + self.head_dim]

        # §4.3 and §4.4 — dot products with scaling
        attn_logits = []
        for t in range(t_plus_1):
            k_t_h = keys_cache[t][head_start : head_start + self.head_dim]
            dot = q_h[0] * k_t_h[0]
            for j in range(1, self.head_dim):
                dot = dot + q_h[j] * k_t_h[j]
            attn_logits.append(dot / sqrt_head_dim)

        # §4.5 — softmax
        attn_weights = softmax(attn_logits)

        # §4.6 — weighted sum
        head_out = []
        for j in range(self.head_dim):
            acc = attn_weights[0] * values_cache[0][head_start + j]
            for t in range(1, t_plus_1):
                acc = acc + attn_weights[t] * values_cache[t][head_start + j]
            head_out.append(acc)
        concat_heads.extend(head_out)

    # An output projection (Linear) comes next — same pattern as W_q.
    return self.output.forward(concat_heads)
```

Every section heading in this handout marks the corresponding block of source code. If you open `model.py` after reading this, the function should read like prose.

Recap in one sentence

Attention takes the current input, projects it three ways, scores each projection's "query" against every cached "key", normalises those scores into a probability distribution with softmax, and returns a weighted mix of the cached "values" — all assembled atom by atom from the same multiply/add/exp/divide operations the perceptron used. Nothing more.